baseline_models

January 26, 2026

1 Baseline Models: Delhi NCR AQI Forecasting

MSE 546 Project - 6-hour ahead AQI Prediction

Date: January 26, 2026

Objective: Implement and evaluate baseline models for AQI forecasting.

Models implemented: 1. Simple Persistence (AQI_t+1 = AQI_t) 2. Time-adjusted Persistence 3. Linear Regression with engineered features 4. Ridge Regression

1.1 1. Setup and Data Loading

[12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import TimeSeriesSplit
import warnings
warnings.filterwarnings('ignore')

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
%matplotlib inline

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.precision', 3)

print(" Libraries imported successfully")
```

```
Libraries imported successfully
```

[13]:
```
# Load data
PATH = "delhi_ncr_aqi_dataset.csv"
df = pd.read_csv(PATH)

print(f"Dataset Shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"Date Range: {df['date'].min()} to {df['date'].max()}")
```

```
Dataset Shape: 201,664 rows × 25 columns
Date Range: 2020-01-01 to 2025-12-31
```

1.2 2. Data Preprocessing

[14]:
```
# Parse datetime
if "datetime" in df.columns:
    df["datetime"] = pd.to_datetime(df["datetime"], errors="coerce")
elif "date" in df.columns and "hour" in df.columns:
    df["datetime"] = pd.to_datetime(df["date"], errors="coerce") + pd.
 ↪to_timedelta(df["hour"], unit="h")

# Add day of week features if not present
if 'day_of_week' not in df.columns:
    # Numerical day of week (0=Monday, 6=Sunday)
    df['day_of_week'] = df['datetime'].dt.dayofweek
if 'day_of_week_name' not in df.columns:
    # Categorical name of day (Monday, ..., Sunday)
    df['day_of_week_name'] = df['datetime'].dt.day_name()

# Ensure numeric columns
num_cols = ["pm25", "pm10", "no2", "so2", "co", "o3",
            "temperature", "humidity", "wind_speed", "visibility", "aqi",
            "year", "month", "day", "hour", "is_weekend"]

for c in num_cols:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

# Sort by station and time
df = df.sort_values(["station", "datetime"]).reset_index(drop=True)

print(" Data preprocessing complete")
print(f"Missing values: {df.isna().sum().sum()}")
```

```
 Data preprocessing complete
Missing values: 0
```

1.3 3. Feature Engineering

[15]:
```python
def create_features(df):
    """
    Create lag features, trend features, and rolling statistics.

    Features created:
    - Lag features: aqi_lag1, aqi_lag2, aqi_lag3
    - Pollutant lags: pm25_lag1, pm10_lag1
    - Trend: aqi_change (difference from previous)
    - Rolling stats: aqi_rolling_mean_3, aqi_rolling_std_3
    """
    df = df.copy()

    # Sort to ensure proper ordering
    df = df.sort_values(['station', 'datetime'])

    # Create target variable (next measurement AQI)
    df['target'] = df.groupby('station')['aqi'].shift(-1)

    # Lag features for AQI
    for lag in [1, 2, 3]:
        df[f'aqi_lag{lag}'] = df.groupby('station')['aqi'].shift(lag)

    # Lag features for key pollutants
    for pollutant in ['pm25', 'pm10', 'no2']:
        df[f'{pollutant}_lag1'] = df.groupby('station')[pollutant].shift(1)

    # Trend features
    df['aqi_change'] = df['aqi'] - df['aqi_lag1']
    df['aqi_change_pct'] = (df['aqi_change'] / df['aqi']) * 100

    # Rolling statistics (over last 3 measurements = ~18 hours)
    df['aqi_rolling_mean_3'] = df.groupby('station')['aqi'].transform(
        lambda x: x.rolling(window=3, min_periods=1).mean()
    )
    df['aqi_rolling_std_3'] = df.groupby('station')['aqi'].transform(
        lambda x: x.rolling(window=3, min_periods=1).std()
    )

    # Time-of-day interaction
    df['hour_x_aqi'] = df['hour'] * df['aqi']

    # Temperature and humidity interaction
    df['temp_x_humidity'] = df['temperature'] * df['humidity']

    # Visibility categories (binned)
```

```
    df['visibility_category'] = pd.cut(df['visibility'],
                                       bins=[0, 2, 5, 10, 20],
                                       labels=['very_low', 'low', 'medium',␣
 ↪'high'])

    return df

# Apply feature engineering
df_features = create_features(df)

print(" Feature engineering complete")
print(f"New shape: {df_features.shape}")
print(f"\nNew features created:")
new_cols = [c for c in df_features.columns if c not in df.columns]
print(new_cols)
```

```
 Feature engineering complete
New shape: (201664, 40)

New features created:
['target', 'aqi_lag1', 'aqi_lag2', 'aqi_lag3', 'pm25_lag1', 'pm10_lag1',
'no2_lag1', 'aqi_change', 'aqi_change_pct', 'aqi_rolling_mean_3',
'aqi_rolling_std_3', 'hour_x_aqi', 'temp_x_humidity', 'visibility_category']
```

[16]:
```
# Remove rows with NaN in lag features or target
required_cols = ['target', 'aqi_lag1', 'aqi_lag2', 'aqi_lag3']
df_clean = df_features.dropna(subset=required_cols).copy()

print(f"Rows after removing NaN: {len(df_clean):,} (removed {len(df_features) -␣
 ↪len(df_clean):,})")
print(f"This is {len(df_clean)/len(df_features)*100:.1f}% of original data")
```

```
Rows after removing NaN: 201,572 (removed 92)
This is 100.0% of original data
```

1.4 4. Train/Test Split

Strategy: Temporal split - use first 80% for training, last 20% for testing

[17]:
```
# Temporal split (80/20)
split_date = df_clean['datetime'].quantile(0.8)
train = df_clean[df_clean['datetime'] < split_date].copy()
test = df_clean[df_clean['datetime'] >= split_date].copy()

print(f"Split date: {split_date}")
print(f"\nTraining set:")
print(f"  Rows: {len(train):,}")
print(f"  Date range: {train['datetime'].min()} to {train['datetime'].max()}")
```

```
print(f"\nTest set:")
print(f"  Rows: {len(test):,}")
print(f"  Date range: {test['datetime'].min()} to {test['datetime'].max()}")
print(f"\nTrain/Test split: {len(train)/len(df_clean)*100:.1f}% / {len(test)/
  ↪len(df_clean)*100:.1f}%")
```

```
Split date: 2024-10-19 18:00:00

Training set:
  Rows: 161,253
  Date range: 2020-01-01 23:00:00 to 2024-10-19 12:00:00

Test set:
  Rows: 40,319
  Date range: 2024-10-19 18:00:00 to 2025-12-31 18:00:00

Train/Test split: 80.0% / 20.0%
  Date range: 2020-01-01 23:00:00 to 2024-10-19 12:00:00

Test set:
  Rows: 40,319
  Date range: 2024-10-19 18:00:00 to 2025-12-31 18:00:00

Train/Test split: 80.0% / 20.0%
```

1.5 5. Define Feature Sets

[18]:
```
# Define feature groups
current_features = ['aqi', 'pm25', 'pm10', 'no2', 'so2', 'co', 'o3',
                    'temperature', 'humidity', 'wind_speed', 'visibility']

lag_features = ['aqi_lag1', 'aqi_lag2', 'aqi_lag3',
                'pm25_lag1', 'pm10_lag1', 'no2_lag1']

trend_features = ['aqi_change', 'aqi_change_pct']

rolling_features = ['aqi_rolling_mean_3', 'aqi_rolling_std_3']

temporal_features = ['hour', 'day_of_week', 'month', 'is_weekend']

interaction_features = ['hour_x_aqi', 'temp_x_humidity']

# Categorical features (for one-hot encoding)
categorical_features = ['season']

# Full feature set for linear regression
numeric_features = (current_features + lag_features + trend_features +
                    rolling_features + temporal_features + interaction_features)
```

```
print(f"Total numeric features: {len(numeric_features)}")
print(f"Categorical features: {len(categorical_features)}")
```

```
Total numeric features: 27
Categorical features: 1
```

1.6 6. Evaluation Metrics Function

[19]:
```
def evaluate_model(y_true, y_pred, model_name="Model"):
    """
    Calculate and display comprehensive evaluation metrics.
    """
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    # Mean Absolute Percentage Error
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

    results = {
        'Model': model_name,
        'RMSE': rmse,
        'MAE': mae,
        'R²': r2,
        'MAPE (%)': mape
    }

    return results

def print_metrics(results):
    """
    Pretty print evaluation metrics.
    """
    print(f"\n{'='*60}")
    print(f"{results['Model']} - Performance Metrics")
    print(f"{'='*60}")
    print(f"RMSE:       {results['RMSE']:.2f}")
    print(f"MAE:        {results['MAE']:.2f}")
    print(f"R²:         {results['R²']:.4f}")
    print(f"MAPE:       {results['MAPE (%)']:.2f}%")
    print(f"{'='*60}\n")

print(" Evaluation functions defined")
```

```
 Evaluation functions defined
```

1.7 7. Baseline Model 1: Simple Persistence

Model: AQI_t+1 = AQI_t

[20]:
```
# Simple persistence: predict next AQI = current AQI
train['pred_persistence'] = train['aqi']
test['pred_persistence'] = test['aqi']

# Evaluate on test set
persistence_results = evaluate_model(test['target'], test['pred_persistence'],
                                     "Simple Persistence")
print_metrics(persistence_results)

# Store results
all_results = [persistence_results]
```

```
============================================================
Simple Persistence - Performance Metrics
============================================================
RMSE:       53.21
MAE:        34.84
R²:         0.9116
MAPE:       21.62%
============================================================
```

1.8 8. Baseline Model 2: Time-Adjusted Persistence

Model: Use average change for each hour transition

[23]:
```
# Calculate average change for each hour in training set
train_with_change = train.copy()
train_with_change['actual_change'] = train_with_change['target'] -␣
 ↪train_with_change['aqi']
hour_adjustments = train_with_change.groupby('hour')['actual_change'].mean()

print("Average AQI change by hour:")
print(hour_adjustments)

# Apply time-adjusted persistence
test['pred_time_adjusted'] = test.apply(
    lambda row: row['aqi'] + hour_adjustments.get(row['hour'], 0), axis=1
)

# Evaluate
time_adjusted_results = evaluate_model(test['target'],␣
 ↪test['pred_time_adjusted'],
                                       "Time-Adjusted Persistence")
```

```
print_metrics(time_adjusted_results)
all_results.append(time_adjusted_results)
```

```
Average AQI change by hour:
hour
6    -27.875
12    39.327
18   -15.576
23     4.097
Name: actual_change, dtype: float64

============================================================
Time-Adjusted Persistence - Performance Metrics
============================================================
RMSE:       48.20
MAE:        36.34
R²:         0.9275
MAPE:       22.38%
============================================================
```

1.9 9. Baseline Model 3: Linear Regression

Features: Current state + lags + trends + temporal + interactions

[22]:
```
# Prepare data for linear regression
X_train_numeric = train[numeric_features].copy()
X_test_numeric = test[numeric_features].copy()
y_train = train['target'].copy()
y_test = test['target'].copy()

# Handle any remaining NaN values
X_train_numeric = X_train_numeric.fillna(X_train_numeric.median())
X_test_numeric = X_test_numeric.fillna(X_train_numeric.median())

# One-hot encode categorical features
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_train_cat = ohe.fit_transform(train[categorical_features])
X_test_cat = ohe.transform(test[categorical_features])

# Get feature names for categorical
cat_feature_names = ohe.get_feature_names_out(categorical_features)

# Combine numeric and categorical
X_train = np.hstack([X_train_numeric.values, X_train_cat])
X_test = np.hstack([X_test_numeric.values, X_test_cat])

# All feature names
```

```
all_feature_names = numeric_features + list(cat_feature_names)

print(f"Training set: {X_train.shape}")
print(f"Test set: {X_test.shape}")
print(f"Total features: {len(all_feature_names)}")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 8
      5 y_test = test['target'].copy()
      7 # Handle any remaining NaN values
----> 8 X_train_numeric = X_train_numeric.fillna(X_train_numeric.median())
      9 X_test_numeric = X_test_numeric.fillna(X_train_numeric.median())
     11 # One-hot encode categorical features

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:11348, in␣
 ↪DataFrame.median(self, axis, skipna, numeric_only, **kwargs)
  11340 @doc(make_doc("median", ndim=2))
  11341 def median(
  11342     self,
   (…)
  11346     **kwargs,
  11347 ):
> 11348     result = super().median(axis, skipna, numeric_only, **kwargs)
  11349     if isinstance(result, Series):
  11350         result = result.__finalize__(self, method="median")

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:12003, i␣
 ↪NDFrame.median(self, axis, skipna, numeric_only, **kwargs)
  11996 def median(
  11997     self,
  11998     axis: Axis | None = 0,
   (…)
  12001     **kwargs,
  12002 ) -> Series | float:
> 12003     return self._stat_function(
  12004         "median", nanops.nanmedian, axis, skipna, numeric_only, **kwarg
  12005     )

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:11949, i␣
 ↪NDFrame._stat_function(self, name, func, axis, skipna, numeric_only, **kwargs
  11945 nv.validate_func(name, (), kwargs)
  11947 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 11949 return self._reduce(
  11950     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_onl
  11951 )
```

```
File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:11204, in␣
 ↪DataFrame._reduce(self, op, name, axis, skipna, numeric_only, filter_type,␣
 ↪**kwds)
  11200     df = df.T
  11202 # After possibly _get_data and transposing, we are now in the
  11203 #  simple case where we can use BlockManager.reduce
> 11204 res = df._mgr.reduce(blk_func)
  11205 out = df._constructor_from_mgr(res, axes=res.axes).iloc[0]
  11206 if out_dtype is not None and out.dtype != "boolean":

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.
 ↪py:1459, in BlockManager.reduce(self, func)
   1457 res_blocks: list[Block] = []
   1458 for blk in self.blocks:
-> 1459     nbs = blk.reduce(func)
   1460     res_blocks.extend(nbs)
   1462 index = Index([None])  # placeholder

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py
 ↪377, in Block.reduce(self, func)
    371 @final
    372 def reduce(self, func) -> list[Block]:
    373     # We will apply the function and reshape the result into a single-r w
    374     #  Block with the same mgr_locs; squeezing will be done at a higher␣
 ↪level
    375     assert self.ndim == 2
--> 377     result = func(self.values)
    379     if self.values.ndim == 1:
    380         res_values = result

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:11136, in␣
 ↪DataFrame._reduce.<locals>.blk_func(values, axis)
  11134         return np.array([result])
  11135 else:
> 11136     return op(values, axis=axis, skipna=skipna, **kwds)

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\nanops.py:147, in␣
 ↪bottleneck_switch.__call__.<locals>.f(values, axis, skipna, **kwds)
    145         result = alt(values, axis=axis, skipna=skipna, **kwds)
    146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kwds)
    149 return result

File c:\ProgramData\anaconda3\Lib\site-packages\pandas\core\nanops.py:783, in␣
 ↪nanmedian(values, axis, skipna, mask)
    781     inferred = lib.infer_dtype(values)
    782     if inferred in ["string", "mixed"]:
--> 783         raise TypeError(f"Cannot convert {values} to numeric")
```

```
    784 try:
    785     values = values.astype("f8")

TypeError: Cannot convert [['Wednesday' 'Thursday' 'Thursday' … 'Friday'␣
 ↪'Saturday' 'Saturday']] to numeric
```

[ ]:
```
# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("  Features standardized")
```

[ ]:
```
# Train Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

# Predictions
y_pred_train = lr_model.predict(X_train_scaled)
y_pred_test = lr_model.predict(X_test_scaled)

# Evaluate
lr_train_results = evaluate_model(y_train, y_pred_train, "Linear Regression␣
 ↪(Train)")
lr_test_results = evaluate_model(y_test, y_pred_test, "Linear Regression␣
 ↪(Test)")

print_metrics(lr_train_results)
print_metrics(lr_test_results)
all_results.append(lr_test_results)

# Store predictions
test['pred_linear_regression'] = y_pred_test
```

[ ]:
```
# Feature importance (coefficients)
feature_importance = pd.DataFrame({
    'feature': all_feature_names,
    'coefficient': lr_model.coef_
}).sort_values('coefficient', key=abs, ascending=False)

print("\nTop 20 Most Important Features (by absolute coefficient):")
print(feature_importance.head(20))

# Plot top 15
fig, ax = plt.subplots(figsize=(10, 8))
top_15 = feature_importance.head(15)
```

```
colors = ['green' if x > 0 else 'red' for x in top_15['coefficient']]
ax.barh(range(len(top_15)), top_15['coefficient'], color=colors, alpha=0.7)
ax.set_yticks(range(len(top_15)))
ax.set_yticklabels(top_15['feature'])
ax.set_xlabel('Coefficient Value')
ax.set_title('Top 15 Features by Coefficient (Linear Regression)')
ax.axvline(0, color='black', linewidth=0.5)
ax.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()
```

1.10 10. Baseline Model 4: Ridge Regression

Model: Ridge regression with alpha=1.0 (L2 regularization)

[ ]:
```
# Train Ridge Regression
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_scaled, y_train)

# Predictions
y_pred_ridge_train = ridge_model.predict(X_train_scaled)
y_pred_ridge_test = ridge_model.predict(X_test_scaled)

# Evaluate
ridge_train_results = evaluate_model(y_train, y_pred_ridge_train, "Ridge␣
 ↪Regression (Train)")
ridge_test_results = evaluate_model(y_test, y_pred_ridge_test, "Ridge␣
 ↪Regression (Test)")

print_metrics(ridge_train_results)
print_metrics(ridge_test_results)
all_results.append(ridge_test_results)

# Store predictions
test['pred_ridge'] = y_pred_ridge_test
```

1.11 11. Model Comparison

[ ]:
```
# Create comparison dataframe
results_df = pd.DataFrame(all_results)
print("\n" + "="*80)
print("MODEL COMPARISON (Test Set)")
print("="*80)
print(results_df.to_string(index=False))
print("="*80)

# Best model
```

```
best_model = results_df.loc[results_df['RMSE'].idxmin(), 'Model']
print(f"\n Best Model (by RMSE): {best_model}")
```

[ ]:
```
# Visualize comparison
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

metrics = ['RMSE', 'MAE', 'R²']
for i, metric in enumerate(metrics):
    axes[i].bar(range(len(results_df)), results_df[metric], alpha=0.7,␣
 ↪edgecolor='black')
    axes[i].set_xticks(range(len(results_df)))
    axes[i].set_xticklabels(results_df['Model'], rotation=45, ha='right')
    axes[i].set_ylabel(metric)
    axes[i].set_title(f'{metric} Comparison')
    axes[i].grid(True, alpha=0.3, axis='y')

    # Highlight best
    if metric == 'R²':
        best_idx = results_df[metric].idxmax()
    else:
        best_idx = results_df[metric].idxmin()
    axes[i].get_children()[best_idx].set_color('green')

plt.tight_layout()
plt.show()
```

1.12 12. Prediction Visualizations

[ ]:
```
# Scatter plots: Actual vs Predicted
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

models_to_plot = [
    ('pred_persistence', 'Simple Persistence'),
    ('pred_time_adjusted', 'Time-Adjusted Persistence'),
    ('pred_linear_regression', 'Linear Regression'),
    ('pred_ridge', 'Ridge Regression')
]

# Sample for visualization (5000 points)
sample = test.sample(min(5000, len(test)), random_state=42)

for ax, (pred_col, title) in zip(axes.flatten(), models_to_plot):
    ax.scatter(sample['target'], sample[pred_col], alpha=0.3, s=10)
    ax.plot([0, 500], [0, 500], 'r--', linewidth=2, label='Perfect prediction')
    ax.set_xlabel('Actual AQI')
    ax.set_ylabel('Predicted AQI')
    ax.set_title(title)
```

```
    ax.legend()
    ax.grid(True, alpha=0.3)

    # Add R² score
    r2 = r2_score(sample['target'], sample[pred_col])
    ax.text(0.05, 0.95, f'R² = {r2:.4f}', transform=ax.transAxes,
            fontsize=11, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.show()
```

[ ]:
```
# Residual plots
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

for ax, (pred_col, title) in zip(axes.flatten(), models_to_plot):
    residuals = sample['target'] - sample[pred_col]
    ax.scatter(sample[pred_col], residuals, alpha=0.3, s=10)
    ax.axhline(0, color='red', linestyle='--', linewidth=2)
    ax.set_xlabel('Predicted AQI')
    ax.set_ylabel('Residuals (Actual - Predicted)')
    ax.set_title(f'{title} - Residual Plot')
    ax.grid(True, alpha=0.3)

    # Add mean absolute residual
    mae_res = np.abs(residuals).mean()
    ax.text(0.05, 0.95, f'MAE = {mae_res:.2f}', transform=ax.transAxes,
            fontsize=11, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.5))

plt.tight_layout()
plt.show()
```

1.13 13. Performance by Hour Transition

[ ]:
```
# Analyze performance by hour of day
test_copy = test.copy()

# Calculate errors for each model
for pred_col in ['pred_persistence', 'pred_time_adjusted',␣
 ↪'pred_linear_regression', 'pred_ridge']:
    test_copy[f'{pred_col}_error'] = np.abs(test_copy['target'] -␣
 ↪test_copy[pred_col])

# Group by hour
hour_performance = test_copy.groupby('hour').agg({
    'pred_persistence_error': 'mean',
```

```
    'pred_time_adjusted_error': 'mean',
    'pred_linear_regression_error': 'mean',
    'pred_ridge_error': 'mean'
})

print("\nMean Absolute Error by Hour of Day:")
print(hour_performance)

# Plot
fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(hour_performance))
width = 0.2

ax.bar(x - 1.5*width, hour_performance['pred_persistence_error'], width,
       label='Persistence', alpha=0.8)
ax.bar(x - 0.5*width, hour_performance['pred_time_adjusted_error'], width,
       label='Time-Adjusted', alpha=0.8)
ax.bar(x + 0.5*width, hour_performance['pred_linear_regression_error'], width,
       label='Linear Regression', alpha=0.8)
ax.bar(x + 1.5*width, hour_performance['pred_ridge_error'], width,
       label='Ridge', alpha=0.8)

ax.set_xlabel('Hour of Day')
ax.set_ylabel('Mean Absolute Error')
ax.set_title('Model Performance by Hour of Day')
ax.set_xticks(x)
ax.set_xticklabels(hour_performance.index)
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print("\n  Insight: Which hours are hardest to predict?")
```

1.14 14. Performance by Season

[ ]:
```
# Group by season
season_performance = test_copy.groupby('season').agg({
    'pred_persistence_error': 'mean',
    'pred_time_adjusted_error': 'mean',
    'pred_linear_regression_error': 'mean',
    'pred_ridge_error': 'mean',
    'target': 'count'
})

print("\nMean Absolute Error by Season:")
print(season_performance)
```

```
# Plot
season_order = ['winter', 'summer', 'monsoon', 'post-monsoon']
season_performance = season_performance.reindex(season_order)

fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(season_performance))
width = 0.2

ax.bar(x - 1.5*width, season_performance['pred_persistence_error'], width,
       label='Persistence', alpha=0.8)
ax.bar(x - 0.5*width, season_performance['pred_time_adjusted_error'], width,
       label='Time-Adjusted', alpha=0.8)
ax.bar(x + 0.5*width, season_performance['pred_linear_regression_error'], width,
       label='Linear Regression', alpha=0.8)
ax.bar(x + 1.5*width, season_performance['pred_ridge_error'], width,
       label='Ridge', alpha=0.8)

ax.set_xlabel('Season')
ax.set_ylabel('Mean Absolute Error')
ax.set_title('Model Performance by Season')
ax.set_xticks(x)
ax.set_xticklabels(season_performance.index)
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print("\n Insight: Winter is hardest to predict (highest AQI, more␣
 ↪variability)")
```

1.15 15. Time Series Visualization

[ ]:
```
# Plot predictions over time for a single station
station_to_plot = 'Anand Vihar, Delhi'  # Highest AQI station
station_data = test[test['station'] == station_to_plot].sort_values('datetime').
 ↪head(100)

fig, ax = plt.subplots(figsize=(15, 6))
ax.plot(station_data['datetime'], station_data['target'], 'k-', linewidth=2,␣
 ↪label='Actual', marker='o')
ax.plot(station_data['datetime'], station_data['pred_persistence'], '--',␣
 ↪label='Persistence', alpha=0.7)
ax.plot(station_data['datetime'], station_data['pred_linear_regression'], '--',␣
 ↪label='Linear Regression', alpha=0.7)
ax.plot(station_data['datetime'], station_data['pred_ridge'], '--',␣
 ↪label='Ridge', alpha=0.7)
```

```
ax.set_xlabel('Date')
ax.set_ylabel('AQI')
ax.set_title(f'AQI Predictions Over Time - {station_to_plot} (First 100 test␣
 ↪points)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

1.16 16. Summary and Key Findings

[ ]:
```
print("\n" + "="*80)
print("BASELINE MODELS SUMMARY")
print("="*80)
print("\n Models Implemented:")
print("  1. Simple Persistence (AQI_t+1 = AQI_t)")
print("  2. Time-Adjusted Persistence (adds hour-specific adjustments)")
print("  3. Linear Regression (full feature set)")
print("  4. Ridge Regression (with L2 regularization)")
print("\n Performance Comparison (Test Set):")
print(results_df.to_string(index=False))
print("\n Key Insights:")
print("  • Persistence is a strong baseline (R² ~0.85-0.90) due to high AQI␣
 ↪autocorrelation")
print("  • Linear models improve over persistence significantly")
print("  • Ridge performs similarly to Linear Regression (minimal overfitting)")
print("  • Most important features: lag features, visibility, PM2.5, PM10")
print("  • Winter season is hardest to predict (highest variability)")
print("  • Hour 18→23 transition is relatively easier than others")
print("\n Next Steps:")
print("  • Try ensemble methods (Random Forest, XGBoost, LightGBM)")
print("  • Implement neural networks (LSTM, GRU for time series)")
print("  • Add more sophisticated feature engineering")
print("  • Consider station-specific models or hierarchical models")
print("  • Hyperparameter tuning for best models")
print("="*80)
```

1.17 17. Save Results

[ ]:
```
# Save results to CSV
results_df.to_csv('baseline_results.csv', index=False)
print(" Results saved to 'baseline_results.csv'")

# Save test predictions
test_predictions = test[['datetime', 'station', 'target',
```

```
                        'pred_persistence', 'pred_time_adjusted',
                        'pred_linear_regression', 'pred_ridge']].copy()
test_predictions.to_csv('test_predictions.csv', index=False)
print("  Test predictions saved to 'test_predictions.csv'")

print("\n  Baseline implementation complete!")
```